

Guía del alumno

Módulo 0, Python básico

Análisis Avanzado de Datos con Python

Cómo usar esta guía

Esta guía es para leer después del laboratorio, no durante. Durante conviene mirar la pantalla y ejecutar, que es donde se aprende. Todo lo que se explicó en voz alta está escrito acá, con el mismo código y las mismas salidas que viste, así que no hace falta tomar apuntes.

Cada tema tiene la misma estructura. Primero qué se vio y para qué sirve. Después el código celda por celda con su explicación. Al final una tabla de referencia, los errores más comunes con el mensaje tal como aparece en pantalla, y tres preguntas para comprobar que quedó claro. Los tres ejercicios están con su solución comentada.

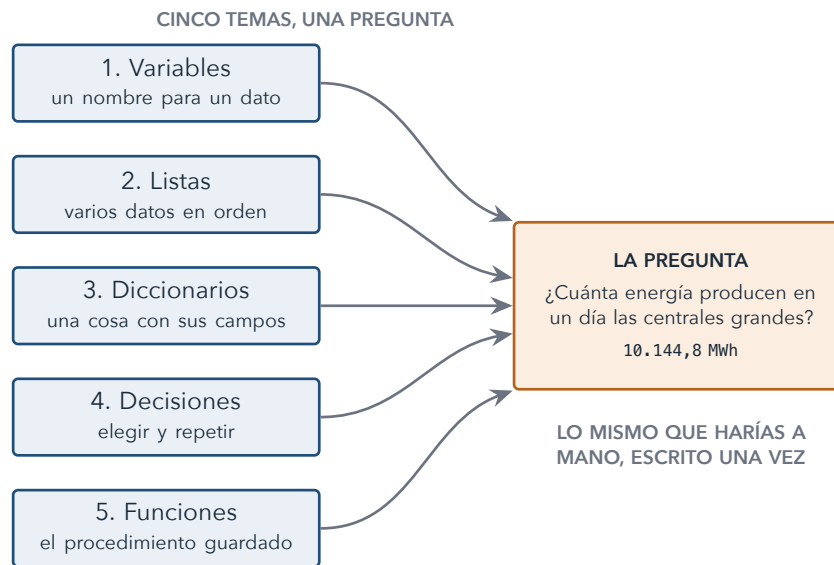
La última página es una hoja de referencia con todo el módulo resumido. Está pensada para imprimir y tenerla al lado durante el resto del curso.

Lo que vas a poder hacer al terminar

- Leer un programa corto de Python y decir qué hace cada línea.
- Guardar un dato con un nombre y saber de qué tipo es.
- Guardar muchos datos juntos y calcular sobre todos de una vez.
- Guardar una cosa con todos sus campos y pedirlos por nombre.
- Escribir una condición que se aplique sola a una lista completa.
- Guardar un procedimiento con un nombre y usarlo cuantas veces quieras.

Todas las salidas de esta guía vienen de la corrida verificada del notebook. Las cinco centrales son inventadas y no tienen relación con las veinte del caso conductor del curso. Se eligieron para que las cuentas den redondas y puedas comprobar a mano lo que hace el programa.

1. El lab en una imagen



La libreta

Todo este lab pasa en una libreta.

Antes de que existiera cualquier sistema, el que estaba a cargo de un grupo de instalaciones llevaba una libreta, y en ella hacía a mano exactamente las cinco cosas de este módulo.

Anotaba un número y al lado le escribía qué era. Hacía una columna con las lecturas de todas las centrales, en el orden de siempre. Tenía una ficha por central, con los campos rotulados. Tenía escrita en el margen una regla, si pasa de tanto es grande. Y cuando el mismo cálculo se repetía todos los días, mandaba a hacer un formulario timbrado.

Nombre y dato es una **variable**. La columna es una **lista**. La ficha es un **diccionario**. La regla del margen es un **if**. El formulario timbrado es una **función**.

Python no inventó nada de esto. Lo único que hace es que la libreta no se canse cuando en vez de cinco centrales son veinte mil.

Las cinco centrales

Son las mismas durante todo el lab y están escritas dentro del notebook, así que no hay que descargar ni abrir ningún archivo.

0.1 La primera celda

```
# Haz clic en esta celda y apriétale Shift+Enter, o el botón de play de la izquierda.
# Estas son las cinco centrales con las que vas a trabajar en todo el lab.
centrales = [
    {"nombre": "Central Río Manso Alto", "tecnologia": "hidro", "potencia_mw": 420,
     ↪ "factor_planta": 0.56},
    {"nombre": "Parque Solar Pampa Alta", "tecnologia": "solar", "potencia_mw": 230,
     ↪ "factor_planta": 0.25},
    {"nombre": "Eólica Cerro Negro", "tecnologia": "eolica", "potencia_mw": 160, "factor_planta":
     ↪ 0.34},
    {"nombre": "Termoeléctrica Bahía Norte", "tecnologia": "gas", "potencia_mw": 375,
     ↪ "factor_planta": 0.5},
    {"nombre": "Diésel Respaldo Cordillera", "tecnologia": "diesel", "potencia_mw": 45,
     ↪ "factor_planta": 0.03},
]
# Estas dos líneas las muestran como tabla. Cómo funcionan lo vas a entender en el tema 3.
for c in centrales:
    print(f'{c["nombre"]:<28}{c["tecnologia"]:<8}{c["potencia_mw"]:>4} MW    factor de planta
     ↪ {c["factor_planta"]}')

```

Central Río Manso Alto	hidro	420 MW	factor de planta 0.56
Parque Solar Pampa Alta	solar	230 MW	factor de planta 0.25
Eólica Cerro Negro	eolica	160 MW	factor de planta 0.34
Termoeléctrica Bahía Norte	gas	375 MW	factor de planta 0.5
Diésel Respaldo Cordillera	diesel	45 MW	factor de planta 0.03

El código de las dos últimas líneas usa cosas del tema 3 y del tema 4. No hace falta entenderlo todavía, y se entiende solo cuando llegues allá.

Estas centrales no existen. Los cinco nombres son inventados y las cifras son verosímiles, no reales. El único dato que se parece al sistema chileno es el factor de planta, que está en el orden de magnitud correcto por tecnología. Nada de este módulo debe citarse como cifra del sector.

2. Tema 1, variables

Qué se vio

Una variable es un nombre pegado a un dato. Se crea con el signo igual, que no quiere decir igual sino *se llama*. A la derecha va el dato y a la izquierda el nombre con el que lo vas a pedir después.

Lo que agrega Python respecto de una planilla es el tipo. No es lo mismo el texto "420" que el número 420, aunque en pantalla se vean casi igual. El primero no se puede sumar y el segundo sí. Esa distinción es la fuente de buena parte de los errores del curso completo, así que conviene tenerla clara desde el principio.

El tipo no se declara. Python lo deduce mirando cómo escribiste el dato.

EL TIPO LO DECIDE PYTHON MIRANDO CÓMO LO ESCRIBISTE

"Central Río Manso Alto"	str	va entre comillas
420	int	entero, sin punto
0.56	float	decimal, con punto
True	bool	verdadero o falso, con mayúscula

Nadie declara el tipo. Python lo deduce de la puntuación, y una comilla o un punto de más cambian lo que después se puede hacer con el dato.

El código, paso a paso

1.1 Cuatro datos con nombre

```
# Una variable se crea con =. A la izquierda va el nombre y a la derecha el dato.
# Estos cuatro datos son de la primera central de la tabla.
nombre = "Central Río Manso Alto" # texto, siempre entre comillas
potencia_mw = 420                 # número entero, la potencia máxima en megawatts
factor_planta = 0.56              # decimal, con punto. Produce en promedio el 56 % de su máximo
renovable = True                  # verdadero o falso, se escribe True o False

# Una f-string arma una frase. Lo que va entre llaves se reemplaza por el valor de la variable.
print(f"La {nombre} tiene {potencia_mw} MW y un factor de planta de {factor_planta}.")
print(f"¿Es renovable? {renovable}")
```

```
La Central Río Manso Alto tiene 420 MW y un factor de planta de 0.56.
¿Es renovable? True
```

Tres detalles que hay que fijar ahora, porque después aparecen como errores incomprensibles.

El texto va **entre comillas**, siempre. El decimal va **con punto** y no con coma, porque en Python la coma separa cosas de una lista, así que 0,56 no es un número sino dos. Y el verdadero se escribe True, con mayúscula inicial.

La última parte de la celda es una *f-string*. La *f* antes de las comillas le avisa a Python que adentro hay llaves con nombres de variables y que hay que reemplazarlas por su valor. Es lo mismo que una plantilla de oficio con los campos por llenar.

1.2 De qué tipo es cada uno

```
# type() dice de qué tipo es un dato. Python lo decide solo, según cómo lo escribiste.
# str es texto, int es entero, float es decimal y bool es verdadero o falso.
print(nombre, "es de tipo", type(nombre))
print(potencia_mw, "es de tipo", type(potencia_mw))
print(factor_planta, "es de tipo", type(factor_planta))
print(renovable, "es de tipo", type(renovable))
```

```
Central Río Manso Alto es de tipo <class 'str'>
420 es de tipo <class 'int'>
0.56 es de tipo <class 'float'>
True es de tipo <class 'bool'>
```

Los nombres cortos que aparecen son los que vas a ver todo el curso. `str` viene de *string*, que es texto. `int` de *integer*, entero. `float` es el decimal. `bool` es verdadero o falso.

Referencia rápida

Instrucción	Para qué sirve
<code>nombre = dato</code>	Crea la variable <code>nombre</code> con ese dato adentro
<code>"texto"</code>	Un texto, siempre entre comillas
<code>420</code>	Un número entero
<code>0.56</code>	Un decimal, con punto
<code>True · False</code>	Verdadero y falso, con mayúscula inicial
<code>type(x)</code>	Dice de qué tipo es <code>x</code>
<code>print(x, y)</code>	Muestra en pantalla, separando con un espacio
<code>f"holá {nombre}"</code>	Arma una frase con el valor de la variable adentro

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>NameError: name 'Potencia' is not defined</code>	Python distingue mayúsculas de minúsculas. <code>Potencia</code> y <code>potencia</code> son dos nombres distintos.
<code>SyntaxError: invalid syntax</code>	Falta una comilla, un paréntesis o el igual está mal puesto. Mira la flecha que Python dibuja debajo de la línea.
<code>TypeError: can only concatenate str (not "int") to str</code>	Estás sumando un texto con un número. Si lo que quieres es armar una frase, usa una <code>f-string</code> .
<code>SyntaxError: invalid decimal literal</code>	Escribiste el decimal con coma. En Python va con punto.

El ejercicio, con su solución

Había que mover el deslizador de potencia_mw a cualquier valor distinto de 420 y volver a correr la celda. La gracia del ejercicio no es la fórmula, es ver que el número de la frase cambia solo cuando cambia el dato, sin tocar la línea del cálculo.

```
potencia_mw = 300
horas = 24
energia_mwh = potencia_mw * horas
print(f"Una central de {potencia_mw} MW operando {horas} horas a plena "
      f"potencia produce {energia_mwh} MWh.")
```

Si la verificación te decía que la potencia seguía en 420, era porque moviste el deslizador pero no volviste a correr la celda. Mover el control no ejecuta nada por sí solo.

Tres preguntas de repaso

1. Qué pasa si le pongo el mismo nombre a dos datos distintos

Se queda con el último y el anterior se pierde, sin ningún aviso. Es el error más común en un notebook largo y es la razón por la que conviene correr las celdas en orden.

2. Por qué el decimal va con punto

Porque la coma en Python separa elementos de una lista. Si escribes 0,56 Python entiende dos datos, el 0 y el 56.

3. Qué diferencia hay entre 420 y "420"

El primero es un número y se puede sumar. El segundo es un texto y no. En pantalla se ven casi igual, y esa es justamente la trampa.

3. Tema 2, listas

Qué se vio

Con cinco centrales alguien podría crear cinco variables y salir del paso. Con veinte ya no, y con veinte mil es absurdo.

Pero el problema de fondo no es la cantidad. Es que si cada dato tiene su propio nombre, cualquier cálculo sobre todos hay que escribirlo tantas veces como datos haya, y el día que llega una central nueva hay que editar el programa. Con una lista el programa no cambia, cambia el dato.

Una lista se escribe entre corchetes, con los datos separados por comas, y mantiene el orden en que los escribiste.

El código, paso a paso

2.1 La lista y las cuatro preguntas

```
# Una lista va entre corchetes, con los datos separados por comas.
# Son las potencias de las cinco centrales, en el mismo orden de la tabla.
potencias = [420, 230, 160, 375, 45]

print("Potencias", potencias)
print("Cantidad de centrales", len(potencias)) # len cuenta los elementos
print("Potencia total", sum(potencias), "MW") # sum los suma
print("La más grande", max(potencias), "MW") # max entrega el mayor
print("La más chica", min(potencias), "MW") # min entrega el menor
```

```
Potencias [420, 230, 160, 375, 45]
Cantidad de centrales 5
Potencia total 1230 MW
La más grande 420 MW
La más chica 45 MW
```

Fíjate en que ninguna de esas cuatro líneas menciona el número cinco. Si mañana son veinte centrales, estas líneas no se tocan. Ese es todo el argumento de las listas.

Un detalle que confunde la primera vez. `len` no entrega la cantidad de megawatts sino la cantidad de elementos de la lista.

2.2 Las posiciones y la primera vuelta

```
# Cada elemento tiene una posición. Python cuenta desde 0, no desde 1.
print("En la posición 0 está", potencias[0], "y en la posición 4 está", potencias[4])

# Un for repite las líneas con sangría una vez por cada elemento de la lista.
# numero parte en 1 y en cada vuelta le sumamos 1, así lleva la cuenta.
numero = 1
for potencia in potencias:
    print("Central", numero, "tiene", potencia, "MW")
    numero = numero + 1
```

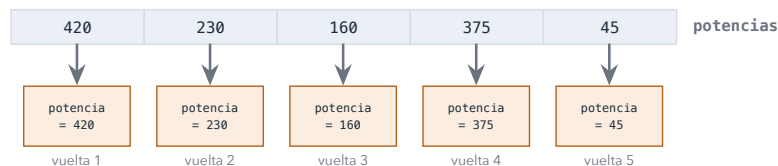
```
En la posición 0 está 420 y en la posición 4 está 45
Central 1 tiene 420 MW
Central 2 tiene 230 MW
Central 3 tiene 160 MW
Central 4 tiene 375 MW
Central 5 tiene 45 MW
```

Acá está lo más raro de Python para quien recién llega. Las posiciones parten en cero, así que la primera central es la cero y la quinta es la cuatro.

La forma que mejor funciona para recordarlo es no pensar en el puesto de la fila sino en cuántos pasos hay que dar desde el principio. Al primero no hay que darle ningún paso.

La segunda parte de la celda es un `for`. Lo importante es qué guarda su variable. No guarda la lista, guarda el elemento de esa vuelta, y por eso se le pone nombre en singular.

UNA VUELTA POR ELEMENTO, Y EN CADA UNA LA VARIABLE CAMBIA



Las líneas con sangría son las que se repiten, y la sangría es lo único que lo dice. Sin ella, Python no sabe dónde termina el `for`.

Referencia rápida

Instrucción	Para qué sirve
<code>[420, 230, 160]</code>	Crea una lista con esos tres datos, en ese orden
<code>lista[0]</code>	El primer elemento, porque se cuenta desde cero
<code>lista[-1]</code>	El último, contando desde el final
<code>len(lista)</code>	Cuántos elementos tiene
<code>sum(lista)</code>	La suma de todos
<code>max(lista) · min(lista)</code>	El mayor y el menor
<code>lista.append(x)</code>	Agrega <code>x</code> al final
<code>for x in lista:</code>	Repite las líneas con sangría, una vuelta por elemento

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>IndexError: list index out of range</code>	Pediste una posición que no existe. Con cinco elementos, la última es la cuatro.
<code>TypeError: unsupported operand type(s) for +: 'int' and 'str'</code>	La lista tiene textos y números mezclados y <code>sum</code> no puede sumarlos.
<code>IndentationError: expected an indented block</code>	Después de los dos puntos del <code>for</code> tiene que venir al menos una línea con sangría.
<code>SyntaxError: invalid syntax</code>	Falta un corchete de cierre, o hay una coma de más al final.

Tres preguntas de repaso

1. Por qué las posiciones parten en cero

Porque la posición es la distancia desde el principio, y el primer elemento está a distancia cero. Todos los lenguajes que vas a encontrar hacen lo mismo.

2. Qué pasa con las cuatro líneas de la celda 2.1 si agrego una sexta central

Nada, siguen funcionando igual y entregan el resultado nuevo. Esa es la razón de usar una lista en vez de cinco variables.

3. Qué guarda la variable del `for` en cada vuelta

El elemento de esa vuelta, no la lista completa. Por eso se le pone nombre en singular.

4. Tema 3, diccionarios

Qué se vio

La lista de potencias sirve para sumar, pero no sirve para responder cuál es la central más grande, porque los nombres se quedaron afuera.

La salida barata sería llevar dos listas en paralelo, una de nombres y otra de potencias, y confiar en que el tercer nombre corresponde a la tercera potencia. Funciona hasta que alguien ordena una de las dos y no la otra. Ahí las cifras quedan cruzadas y nadie se entera, porque el programa no reclama.

Un diccionario resuelve eso juntando los campos de una misma cosa. Se escribe entre llaves, y cada dato lleva su nombre al lado, que se llama clave.

Y una lista de diccionarios es, literalmente, una tabla. Cada diccionario es una fila y cada clave es una columna. Es la estructura con la que vas a trabajar todo el curso, y en el Módulo 1 va a aparecer otra vez con el nombre de DataFrame.

El código, paso a paso

3.1 Una ficha y sus campos

```
# centrales es una lista de diccionarios. Con [0] tomamos el primero.
central = centrales[0]
print(central)

# En un diccionario cada dato se pide por su nombre, que se llama clave.
print("Nombre      ", central["nombre"])
print("Tecnología  ", central["tecnologia"])
print("Potencia    ", central["potencia_mw"], "MW")
print("Factor      ", central["factor_planta"])
```

```
{'nombre': 'Central Río Manso Alto', 'tecnologia': 'hidro', 'potencia_mw': 420, 'factor_planta': 0.56}
Nombre      Central Río Manso Alto
Tecnología  hidro
Potencia    420 MW
Factor      0.56
```

Llaves para el diccionario, corchetes para la lista. Es la única regla de puntuación del tema.

La diferencia con la lista está en cómo se pide el dato. En una lista se pide por posición, que es un número. En un diccionario se pide por clave, que es una palabra, y va entre comillas porque es texto.

3.2 El archivador completo

```
# El for recorre la lista. En cada vuelta, central es un diccionario distinto.
# Así funciona la tabla de la primera celda.
for central in centrales:
    print(central["nombre"], "tiene", central["potencia_mw"], "MW")
```

```
Central Río Manso Alto tiene 420 MW
Parque Solar Pampa Alta tiene 230 MW
Eólica Cerro Negro tiene 160 MW
Termoeléctrica Bahía Norte tiene 375 MW
Diésel Respaldo Cordillera tiene 45 MW
```

Esta es la tabla de la primera celda del lab, la que corriste sin entender. El `for` recorre la lista y en cada vuelta la variable `central` es un diccionario distinto.

Referencia rápida

Instrucción	Para qué sirve
<code>{"nombre": "Río Manso"}</code>	Crea un diccionario con una clave y su valor
<code>d["nombre"]</code>	El valor guardado bajo esa clave
<code>d["nuevo"] = 5</code>	Agrega la clave si no estaba, o cambia su valor si ya estaba
<code>lista[0]["nombre"]</code>	Un campo del primer diccionario de la lista
<code>for d in lista:</code>	Recorre la lista, una vuelta por diccionario
<code>lista.append(d)</code>	Agrega un diccionario al final

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>KeyError: 'potencia'</code>	Esa clave no existe. Suele ser el nombre incompleto, en este lab la clave es <code>potencia_mw</code> .
<code>KeyError: 'tecnología'</code>	Una tilde de diferencia. Las claves de este lab van sin tilde a propósito, por esto mismo.
<code>TypeError: list indices must be integers or slices, not str</code>	Le pediste una clave a la lista en vez de al diccionario. Falta el <code>[0]</code> del medio.
<code>NameError: name 'centrales' is not defined</code>	No corriste la primera celda del notebook, o corriste las celdas fuera de orden. Se arregla con Entorno de ejecución, Reiniciar y ejecutar todo.

El ejercicio, con su solución

Había que completar dos huecos para quedarse con los nombres de las centrales de más de 200 megawatts.

3.3 El ejercicio, como venía

```
# Tu turno 2
# Queremos los nombres de las centrales de más de 200 MW. Completa los dos huecos.
seleccionadas = []          # lista vacía, aquí se van juntando los nombres
for central in centrales:
    # Hueco 1, reemplaza ____ por la potencia de esta central
    if central["potencia_mw"] > 200:          # if deja pasar solo lo que cumple la
        ↪ condición, lo verás en el tema 4
        # Hueco 2, reemplaza ____ por el nombre de esta central
        seleccionadas.append(central["nombre"]) # append agrega un dato al final de la lista
print("Centrales de más de 200 MW", seleccionadas)
```

```
Centrales de más de 200 MW ['Central Río Manso Alto', 'Parque Solar Pampa Alta', 'Termoeléctrica Bahía Norte']
```

En el primer hueco va la potencia de la central que se está mirando en esa vuelta, y en el segundo va su nombre. Los dos se piden igual que en la celda 3.1, con la clave entre corchetes.

```
seleccionadas = []
for central in centrales:
    if central["potencia_mw"] > 200:
        seleccionadas.append(central["nombre"])
print("Centrales de más de 200 MW", seleccionadas)
```

La línea con ____ igual a cero, que estaba arriba, desaparece cuando completas el ejercicio. Estaba solo para que la celda pudiera correr sin completar.

Fíjate en el patrón, porque se repite en todas partes. Lista vacía, un for que recorre, una condición que filtra y un append que junta. En el Módulo 2 eso mismo se escribe en una línea, pero por dentro hace esto.

Tres preguntas de repaso

1. Importa el orden de los campos de un diccionario

No. En una lista la posición lo es todo, en un diccionario no significa nada. Dos fichas con los mismos campos en otro orden son la misma ficha.

2. Cómo pedirías el factor de planta de la tercera central

Con `centrales[2]["factor_planta"]`. El dos es la tercera porque se cuenta desde cero.

3. Por qué las claves de este lab van sin tilde

Porque una tilde de diferencia es un `KeyError`, y en el trabajo real los nombres de columna llegan escritos de cualquier forma. Es una convención que se paga sola.

5. Tema 4, decisiones y repetición

Qué se vio

Todo informe clasifica. Grande y chica, dentro y fuera de norma, al día y atrasada. Y esa clasificación casi nunca viene en el dato, la pone quien analiza.

Con `if`, `elif` y `else` el criterio queda escrito, y escrito se puede discutir, revisar y corregir. Dentro de un `for`, además, se aplica igual a las cinco centrales o a las veinte mil.

Hay dos cosas que conviene fijar. La primera es que Python revisa las condiciones en orden y ejecuta solo la primera que se cumple. La segunda es que la sangría no es estética, es la que dice qué líneas dependen de cada condición.

El código, paso a paso

4.1 La regla, con deslizador

```
# Mueve el deslizador y vuelve a correr la celda para ver cómo cambia la categoría.
potencia = 230 #@param {type:"slider", min:0, max:500, step:10}

# Python revisa las condiciones en orden y ejecuta solo la primera que se cumple.
# Las líneas con sangría son las que dependen de cada condición.
if potencia < 100:
    categoria = "chica"           # menos de 100 MW
elif potencia < 300:
    categoria = "mediana"        # desde 100 y hasta menos de 300 MW
else:
    categoria = "grande"         # 300 MW o más
print("Una central de", potencia, "MW es", categoria)
```

Una central de 230 MW es mediana

El segundo `elif` no necesita decir que además la potencia es mayor que 100. Si el programa llegó hasta ahí es porque la primera condición no se cumplió, así que ya lo sabemos.

Esa es la razón por la que el orden importa. Si pusieras primero la condición de 300, ninguna central caería nunca en mediana.

4.2 El mismo criterio para las cinco

```
# El mismo criterio para las cinco centrales, contando cuántas caen en cada categoría.
conteo = {"chica": 0, "mediana": 0, "grande": 0} # un contador por categoría, todos parten en 0
for central in centrales:
    if central["potencia_mw"] < 100:
        categoria = "chica"
    elif central["potencia_mw"] < 300:
        categoria = "mediana"
    else:
        categoria = "grande"
    print(central["nombre"], "es", categoria)
    conteo[categoria] = conteo[categoria] + 1 # suma 1 al contador de su categoría
print("Conteo final", conteo)
```

```
Central Río Manso Alto es grande
Parque Solar Pampa Alta es mediana
Eólica Cerro Negro es mediana
Termoeléctrica Bahía Norte es grande
Diésel Respaldo Cordillera es chica
Conteo final {'chica': 1, 'mediana': 2, 'grande': 2}
```

Acá está junto todo lo que llevamos. Una lista, cinco diccionarios, un for que los recorre y un if que decide.

Y hay algo nuevo que conviene mirar. El diccionario conteo no guarda una central, guarda tres contadores, uno por categoría. Usar un diccionario para contar es una de las cosas más frecuentes que vas a hacer, y en el Módulo 2 se resuelve con una sola línea que se llama `value_counts`.

El corte lo eligió quien analiza. Los cortes en 100 y en 300 megawatts no salen de ninguna norma ni de ningún reglamento. Se eligieron para este lab.

Una central de 290 megawatts es mediana con este corte y sería grande con el corte en 250, y el informe diría otra cosa. La regla que se lleva el curso es simple. Cuando el criterio lo pone quien analiza, el criterio va escrito en el informe.

Referencia rápida

Instrucción	Para qué sirve
<code>if condicion:</code>	Ejecuta lo de abajo si la condición se cumple
<code>elif otra:</code>	Se revisa solo si las anteriores no se cumplieron
<code>else:</code>	Lo que pasa si no se cumplió ninguna
<code>== · !=</code>	Igual a, distinto de
<code>< · <= · > · >=</code>	Menor, menor o igual, mayor, mayor o igual
<code>and · or · not</code>	Combina condiciones
<code>d[clave] = d[clave] + 1</code>	Suma uno al contador de esa clave

Errores comunes

Lo que aparece en pantalla	Qué pasó
SyntaxError: expected ':'	Falta el signo de dos puntos al final de la línea del <code>if</code> o del <code>for</code> .
IndentationError: unindent does not match any outer indentation level	Mezclaste sangrías de distinto largo. Usa siempre cuatro espacios.
SyntaxError: invalid syntax en un <code>if</code>	Escribiste un solo igual en vez de dos. Uno asigna, dos comparan.
No da error pero el resultado está mal	Suele ser el orden de las condiciones, o una línea que quedó fuera de la sangría del <code>for</code> .

Tres preguntas de repaso

1. Qué pasa si pongo primero la condición de 300

Que ninguna central caería nunca en mediana, porque toda central de más de 300 ya habría entrado en la primera condición. El orden de un `if` encadenado es parte del criterio.

2. Es obligatorio el `else`

No. Si no lo pones y no se cumple ninguna condición, simplemente no pasa nada. A veces es lo que quieres y a veces es un error que no reclama, así que conviene ponerlo.

3. Por qué 100 y 300 y no otros valores

Porque los elegimos para este lab, no hay norma detrás. Esa es justamente la advertencia del tema.

6. Tema 5, funciones

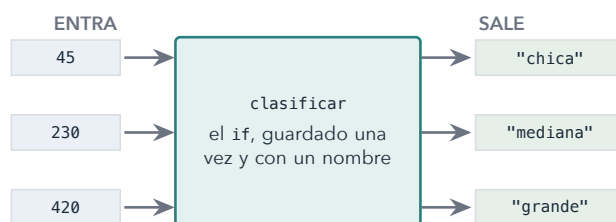
Qué se vio

En los temas anteriores el mismo criterio de clasificación se escribió dos veces. Con dos veces todavía se puede vivir.

El problema aparece cuando el criterio cambia. Hay que acordarse de los dos lugares, y si uno se olvida, el informe queda con dos criterios distintos adentro y no hay forma de darse cuenta mirando el resultado. Esto pasa de verdad, y pasa en planillas que llevan años circulando.

Una función es la respuesta. El procedimiento queda escrito en un solo lugar, con un nombre, y todos los usos apuntan ahí. Cuando cambia, cambia una vez.

ESCRITO UNA VEZ, USADO TRES



Si mañana el corte de central grande cambia de 300 a 250, se corrige adentro de la función y queda corregido en los tres usos. Esa es toda la razón por la que existen.

El código, paso a paso

5.1 La función que clasifica

```
# def crea la función. potencia es el parámetro, el dato que recibe cada vez que la llamas.
def clasificar(potencia):
    if potencia < 100:
        return "chica"          # return entrega el resultado y termina la función
    elif potencia < 300:
        return "mediana"
    else:
        return "grande"

# Al llamarla, el valor entre paréntesis ocupa el lugar de potencia.
print("45 MW es", clasificar(45))
print("230 MW es", clasificar(230))
print("420 MW es", clasificar(420))
```

```
45 MW es chica
230 MW es mediana
420 MW es grande
```

def se lee como definir. Después va el nombre, que lo eliges tú, y entre paréntesis el hueco que la función espera que le llenen. Ese hueco se llama parámetro.

Una cosa que confunde la primera vez. Definir la función no hace nada. Solo la deja guardada. Recién cuando la llamas, con el nombre y un valor entre paréntesis, se ejecuta lo de adentro.

return entrega el resultado y termina la función ahí mismo. No es lo mismo que print. print

muestra algo en pantalla y ahí se acaba, mientras que `return` entrega un dato que puedes guardar en una variable y seguir calculando. Una función que solo imprime casi nunca sirve.

5.2 Dos parámetros

```
# Una función puede recibir más de un parámetro, separados por coma.
# En un día de 24 horas, la central produce su potencia por el factor de planta.
def energia_dia(potencia_mw, factor_planta):
    return potencia_mw * 24 * factor_planta    # el resultado queda en MWh

# La probamos con la primera central, pasándole sus datos desde el diccionario.
primera = centrales[0]
print(primera["nombre"], "produce", energia_dia(primera["potencia_mw"], primera["factor_planta"]),
      ↪ "MWh en un día")
```

Central Río Manso Alto produce 5644.8 MWh en un día

Dos parámetros separados por coma, y el orden importa. Si los pasas al revés el programa no reclama, entrega un número que está mal. Ese tipo de error es el más difícil de encontrar, y por eso conviene que los nombres digan qué son.

La llamada saca los dos datos de la ficha de la central, con sus claves. Acá se juntan el diccionario del tema 3 y la función del tema 5.

Referencia rápida

Instrucción	Para qué sirve
<code>def nombre(parametro):</code>	Define la función. Lo de abajo va con sangría
<code>return valor</code>	Entrega el resultado y termina la función
<code>nombre(dato)</code>	La llama, y el dato ocupa el lugar del parámetro
<code>def f(a, b):</code>	Dos parámetros, y el orden en que los pasas importa
<code>x = nombre(dato)</code>	Guarda en x lo que la función devolvió

Errores comunes

Lo que aparece en pantalla	Qué pasó
<code>NameError: name 'clasificar' is not defined</code>	No corriste la celda que la define, o la definiste más abajo de donde la usas.
<code>TypeError: clasificar() missing 1 required positional argument</code>	La llamaste sin darle el dato que espera, con los paréntesis vacíos.
<code>TypeError: clasificar() takes 1 positional argument but 2 were given</code>	Le pasaste un dato de más.
La función imprime pero no se puede sumar	Usaste <code>print</code> en vez de <code>return</code> . <code>print</code> muestra, <code>return</code> entrega.

El ejercicio, con su solución

La pregunta del lab. ¿Cuánta energía producen en un día las centrales grandes?

5.3 Tu turno, la pregunta del lab

```
# Tu turno 3
# La pregunta del lab. ¿Cuánta energía producen en un día las centrales grandes?
# Escribe tu código acá abajo, siguiendo los cuatro pasos de la caja naranja.
total = 0
for central in centrales:
    if clasificar(central["potencia_mw"]) == "grande":
        total = total + energia_dia(central["potencia_mw"], central["factor_planta"])
print("Las centrales grandes producen", round(total, 1), "MWh en un día")
```

Las centrales grandes producen 10144.8 MWh en un día

5.4 La verificación

```
# Verificación del ejercicio 3. Corre esta celda después de correr tu código.
# globals() revisa si la variable total ya existe.
if "total" not in globals():
    print("Todavía no existe la variable total. Escribe tu código arriba y córrelo.")
elif round(total, 1) == 10144.8:
    print("Correcto. Las centrales grandes producen", round(total, 1), "MWh en un día.")
elif round(total, 1) == 12862.8:
    print("Sumaste las cinco centrales. Revisa el if, solo cuentan las grandes.")
elif total == 795:
    print("Sumaste potencias y no energía. Usa energia_dia para cada central grande.")
else:
    print("Todavía no. Revisa que sumes energia_dia de cada central grande.")
```

Correcto. Las centrales grandes producen 10144.8 MWh en un día.

Los cuatro pasos son siempre los mismos. Una variable que parte en cero, un for que recorre, un if que filtra y una suma que acumula. Es el mismo patrón del ejercicio anterior, con una suma en vez de un append.

Dos errores típicos, y los dos tienen mensaje propio en la verificación. Si te dio 12.862,8 es que sumaste las cinco centrales y te faltó el if. Si te dio 795 es que sumaste las potencias en vez de las energías, o sea que te faltó usar `energia_dia`.

Esa cifra salió de dos de las cinco centrales, las dos que el criterio dejó como grandes. No la compares con el sistema real, que acá los datos son inventados. Lo que importa es que ningún número está escrito a mano, todos los calculó el programa a partir de la lista.

Tres preguntas de repaso

1. Por qué return y no print

Porque print muestra en pantalla y ahí se acaba, y return entrega un dato que se puede guardar y seguir usando. El ejercicio no funcionaría con print, porque hay que sumar lo que la función devuelve.

2. Dónde hay que escribir la función, arriba o abajo de donde se usa

Antes de usarla. Python ejecuta las celdas en el orden en que tú las corres, así que si llamas a una función que todavía no definiste, da error de nombre no encontrado.

3. Qué habría que cambiar para que el corte de central grande fuera 250

Un solo número, el 300 de adentro de clasificar. Y el total del ejercicio cambiaría solo, sin tocar nada más. Esa es toda la razón por la que existen las funciones.

Python básico en una plana

Para imprimir y tener al lado durante el resto del curso

Los cinco temas

Tema	Se escribe	Para qué
Variable	nombre = dato	Ponerle un nombre a un dato
Lista	[a, b, c]	Varios datos en orden, en una sola variable
Diccionario	{"clave": valor}	Los campos de una sola cosa, rotulados
Decisión	if · elif · else	Elegir qué hacer según una condición
Repetición	for x in lista:	Una vuelta por elemento
Función	def nombre(p):	Un procedimiento guardado con un nombre

Lo que se usa todos los días

len(x)	Cuántos elementos tiene
sum(x) · max(x) · min(x)	Suma, mayor y menor
lista[0]	El primero, porque se cuenta desde cero
lista[-1]	El último
lista.append(x)	Agrega al final
d["clave"]	El valor guardado bajo esa clave
type(x)	De qué tipo es
round(x, 1)	Redondea a un decimal
f"texto {variable}"	Arma una frase con el valor adentro

Las cinco trampas de la primera semana

Las posiciones parten en cero	La quinta central es lista[4]
El decimal va con punto	0.56, nunca 0,56
Mayúsculas y minúsculas importan	Potencia no es potencia
La sangría es sintaxis	Cuatro espacios, y siempre los mismos
Un igual asigna, dos comparan	x = 3 guarda, x == 3 pregunta

El patrón que se repite en todo el curso

```
resultado = []           # o un cero, si lo que se va a hacer es sumar
for cosa in coleccion:   # una vuelta por elemento
    if condicion:         # se queda solo con las que cumplen
        resultado.append(algo) # o resultado = resultado + algo
print(resultado)
```

Desde el Módulo 1 en adelante esto se escribe en una línea con pandas. Por dentro sigue haciendo exactamente esto, y el día que algo salga raro, saber esto es lo que te deja entender qué pasó.